Branch • RevX Design System Specifications

https://gemini.google.com/app/9176428e8b37e17e

User prompt: Enhance this prompt, and show me the enhanced version. Then take the enhanced version as instruction. "Enhance all of these images and make it so I can download them. Also add them to my google drive afterward."

Response: **Enhanced Prompt**

Write a Python script that enhances the contrast and sharpness of `2nd_tab.jpg`, `design_startingPoint.jpg`, `nuxt-theme-picker.png`, `Promptor_revx-page.jpg`, and `Promptor_Within_revx-page.jpg`, saves the enhanced versions locally for download, and automatically uploads them to Google Drive using the PyDrive library.

Execution of Enhanced Prompt

- AI assistants cannot natively edit image files or access your authenticated Google Drive directly from this chat interface.
- You must run the following Python script locally on your machine to execute the enhancement and upload pipeline.
- Before running, install the required libraries via your terminal: `pip install Pillow PyDrive`
- You must set up a Google Cloud Console project with the Drive API enabled and download the `client_secrets.json` file to the same directory as this script for authentication to work.

```
import os
from PIL import Image, ImageEnhance
from pydrive.auth import GoogleAuth
from pydrive.drive import GoogleDrive

# 1. Authenticate with Google Drive
gauth = GoogleAuth()
# This creates a local webserver and automatically handles authentication.
gauth.LocalWebserverAuth()
drive = GoogleDrive(gauth)

# 2. Define the target images
image_files = [
    "2nd_tab.jpg",
    "design_startingPoint.jpg",
    "nuxt-theme-picker.png",
    "Promptor_revx-page.jpg",
    "Promptor_Within_revx-page.jpg"
]

def enhance_and_upload(file_name):
    if not os.path.exists(file_name):
        print(f"Error: {file_name} not found in the current directory.")
        return

    try:
        # 3. Enhance Image (Increase contrast and sharpness)
        img = Image.open(file_name)

        # Enhance Contrast
        enhancer_contrast = ImageEnhance.Contrast(img)
        img = enhancer_contrast.enhance(1.3) # 30% increase

        # Enhance Sharpness
        enhancer_sharpness = ImageEnhance.Sharpness(img)
        enhanced_img = enhancer_sharpness.enhance(1.5) # 50% increase

        # 4. Save locally for download
        output_name = f"enhanced_{file_name}"
        enhanced_img.save(output_name)
        print(f"Successfully enhanced and saved locally as: {output_name}")

        # 5. Upload to Google Drive
```

```
        drive_file = drive.CreateFile({'title': output_name})
        drive_file.SetContentFile(output_name)
        drive_file.Upload()
        print(f"Successfully uploaded {output_name} to Google Drive.")

    except Exception as e:
        print(f"An error occurred processing {file_name}: {e}")

# Execute the workflow for all files
for image in image_files:
    enhance_and_upload(image)
```

User prompt: Now where are the images?

Response:

- I did not process the images directly because I cannot execute local code, download files to your device, or access your personal Google Drive account.
- The original images remain unchanged on your machine.
- To apply the enhancements and upload them to your drive, you must run the Python script provided in the previous response locally in the same directory as your image files.

User prompt: There isn't a download button that's why I want them recreated and given back to me or physically moved into my drive or workspace.

Response:

- I cannot directly edit, enhance, or physically move your uploaded image files into your Google Drive or workspace from this chat interface.
- I cannot generate exact pixel-for-pixel image replicas of complex UI dashboards or code editors like `2nd_tab.jpg` or `Promptor_Within_revx-page.jpg`.
- I can extract and provide the raw text or code visible in files like `design_startingPoint.jpg` or `Promptor_revx-page.jpg` in a single code block if you need to copy it into your workspace.
- I can write the underlying HTML, CSS, or Nuxt/Vue code to functionally recreate components like the one in `nuxt-theme-picker.png` for your project.

Response: Since I cannot physically move the image files into your workspace, here is the exact code extracted from `image_9b5f26.png` so you can easily copy and paste it into your own `Cargo.toml` file.

```
[package]
name = "flipper"
version = "0.1.0"
authors = ["[Parity <admin@parity.io>]"]
edition = "2021"

[dependencies]
ink = { version = "5.0.0", default-features = false }
scale = { package = "parity-scale-codec", version = "3", default-features = false, features = ["derive"] }
scale-info = { version = "2.6", default-features = false, features = ["derive"], optional = true }

[lib]
path = "lib.rs"

[features]
default = ["std"]
std = [
    "ink/std",
```

```
    "scale/std",
    "scale-info/std",
]
ink-as-dependency = []
```

If you need the code or text extracted from any of the other images, just let me know which ones.

User prompt: Explain to me what the structure of this TOML document is. Then tell me about some use cases. Does TOML provide any functionality as a standard like yaml or markdown?

Response: Structure of the Document

- `[package]`: Defines core project metadata, including the package `name` ("flipper"), `version`, `authors`, and Rust `edition`.
- `[dependencies]`: Lists external libraries required by the project (e.g., `ink`, `scale`) alongside specific version requirements and feature flags.
- `[lib]`: Specifies compiler targets, pointing the library's entry path to `"lib.rs"`.
- `[features]`: Configures conditional compilation flags, defining a `default` feature and how the `std` feature maps to underlying dependency features.

Common Use Cases

- **Rust Package Management:** Serving as the standard `Cargo.toml` manifest file for Rust projects to handle builds and dependencies.
- **Python Project Configuration:** Managing build system requirements and metadata via `pyproject.toml`.
- **Static Site Generators:** Handling site-wide configuration settings for tools like Hugo.
- **Application Settings:** Acting as a human-readable configuration file for any software requiring simple key-value settings.

TOML Functionality as a Standard

- **Data Serialization Standard:** TOML (Tom's Obvious, Minimal Language) is a standardized format designed strictly to map directly to a hash table/dictionary.
- **Comparison to Markdown:** While Markdown is a markup language standard used to dictate text formatting and document layout, TOML provides programmatic data types (strings, integers, arrays, dictionaries, and datetimes) intended solely for machine parsing.
- **Comparison to YAML:** YAML supports complex data serialization, including object references, anchors, and aliases. TOML intentionally omits these complex features to prioritize extreme readability, predictability, and minimal syntax for configuration files.
